Forge: Zero-Trust Context Compilation, Predictive Sandboxing, and Cryptographic Attestation for High-Assurance LLM Workflows

Jacob Sorel Ferion

Independent Systems Architect • jacobferion@gmail.com

Abstract. The barrier to autonomous Large Language Model (LLM) deployment in enterprise operations is not capability but liability. An agent that writes a hallucinated payload to a production system converts a model error into an irreversible business event: the expenditure is incurred, the system of record is corrupted, and no artifact exists that an auditor, counterparty, or regulator can independently check. Detection after the write is a reporting improvement, not a control. Interpretable Context Methodology (ICM) [7] established basic auditability by treating the filesystem as the operational interface for agent loops, but practical deployments revealed three residual failure modes: the absence of formal compiler discipline, stochastic evaluation vulnerabilities, and the inability to evidence a run's conformance to policy to third parties. We present *Forge*, an AI hypervisor architecture founded on Deterministic Context Compilation (DCC), Calibrated Psychometric Gate Governance (CPG), and zero-trust cryptographic attestation, which relocates enforcement from post-hoc inspection to pre-execution admission control. Transitioning from passive filesystem orchestration to an active, headless Model Context Protocol (MCP) server, Forge formalises multi-stage LLM pipelines through: *(i)* predictive simulation sandboxing, in which input-seeded deterministic dry runs (`simulate_execution`) block hallucinated CRM and API payloads before production expenditure; *(ii)* physical edit-surface isolation with transitive Merkle staleness, eliminating artifact mutation and enabling minimal blast-radius incremental compilation; *(iii)* effort-calibrated multi-sample gate evaluation, using variance-as-signal halts (E_UNSTABLE_GATE) to distinguish output defects from rubric ambiguity; and *(iv)* asymmetric cryptographic receipts, emitting Ed25519-signed attestations (`forge://run/{run_id}/certificate`) that allow a counterparty to verify offline, and without access to private signing material, that a given run cleared its declared gates and has not been altered since. We define the theoretical framework of DCC, articulate the structural invariants governing Forge's MCP architecture, and evaluate its execution properties across database-backed execution proofs. The practical consequence is an execution path in which no production write occurs without a deterministic rehearsal, a bounded evaluation, and a durable record of the authority under which it proceeded.

Keywords. LLM orchestration; deterministic compilation; agent governance; MCP; evaluation variance; cryptographic attestation; zero-trust architecture.

1 Introduction and Background

The deployment of Large Language Models across high-assurance domains, including enterprise operations, automated healthcare triage, and CRM routing, is constrained by the non-deterministic, black-box nature of multi-agent state machines. Traditional agent frameworks store intermediate state in transient memory, execute API calls unconstrained, and lack deterministic replayability.

While ICM [7] addressed initial opacity by using materialised filesystem artifacts, modern LLM operations require active governance. Relying on an agent to police its own output, or on a human to review dashboards manually, creates an unscalable liability bottleneck. Standard evaluation metrics compound the problem: an agent's claim that an action is safe

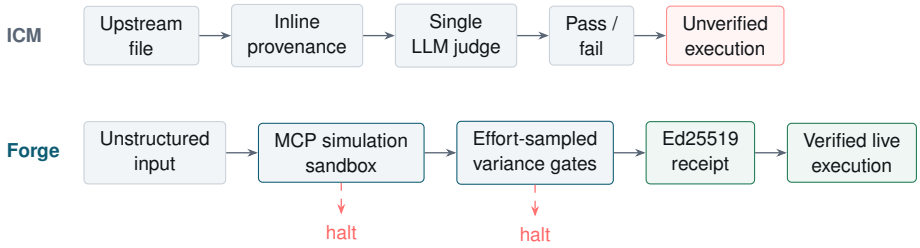

Figure 1. Control flow under ICM and under Forge. ICM verifies after the fact; Forge interposes a deterministic sandbox and a sampled gate before any live write, and both stages are permitted to halt the run.

carries no evidence an external stakeholder, such as a clinic owner or an auditor, can check independently.

Forge addresses these failure modes by upgrading LLM orchestration into a formal, headless compliance hypervisor. Operating over the Model Context Protocol (MCP) [6], Forge acts as a zero-trust gateway [4]: agents must submit proposed actions for deterministic simulation, pass strict psychometric variance gates, and receive a cryptographically signed receipt before any production database is touched.

2 Gaps in Interpretable Context Methodology

Figure 1 contrasts the two control flows. The distinction is where authority sits: ICM grants execution authority to the agent and inspects afterwards, while Forge withholds it until a deterministic check has cleared.

2.1 Coarse Invalidation and Metadata Pollution

ICM tracks workflow progress coarsely. Modifying an upstream specification forces all-or-nothing re-execution. Injecting machine metadata directly into generated files further corrupts byte-level determinism and breaks downstream parsing.

2.2 The Epistemic Fallacy of Single-Sample Judges

Standard LLM-as-a-judge frameworks [5] deploy arbitrary score thresholds without empirical predictive grounding. Evaluating an artifact through a single inference pass ignores judge variance. A threshold of 9.5 applied to the sample set $\{9.0, 9.8, 9.1\}$ yields a non-deterministic pass/fail state that reflects sample noise rather than artifact quality.

2.3 The Blind Execution Liability

Agents in traditional pipelines execute external API calls, such as CRM updates, on internal logic alone. When a hallucination occurs, the cost is already spent and the production record is already corrupted. Detection after the write is a reporting improvement, not a control.

3 The Forge Methodology: Deterministic Context Compilation

Forge redefines orchestration through a headless MCP architecture and a small set of strict physical primitives.

3.1 Predictive Simulation Sandboxing

Forge intercepts agent intent prior to live execution. As an MCP server it exposes a `simulate_execution` tool with the following contract:

- The agent proposes an action payload.
- Forge instantiates an ephemeral in-memory sandbox, mocking external dependencies.
- **Deterministic seeding.** To prevent an agent from repeatedly calling the simulation to brute-force a passing stochastic score, the simulation's variance profile is deterministically seeded from the input payload. Identical inputs yield identical scores.
- The tool returns a `SimulationResult` carrying predicted cost and compliance scores, halting the agent when violation_probability $=$ true.

3.2 Transitive Per-File Merkle Staleness Tracking

Staleness is tracked with hash-chained digests in the sense of Merkle [1].

Let $O_{i,j}$ be the j-th output file produced by stage S_i, and let $C_k \subset \bigcup_{m<k}\{O_{m,*}\}$ be the explicit upstream inputs to stage S_k. Forge computes the input digest

$$\mathcal{D}(S_k) \;=\; \bigoplus_{f \in C_k} \mathrm{SHA256}(f) \;\oplus\; \bigoplus_{s \in \mathrm{Standards}(S_k)} \mathrm{SHA256}(s). \tag{1}$$

A step S_k is marked `stale: true` if and only if $\mathcal{D}_{\text{current}}(S_k) \neq \mathcal{D}_{\text{persisted}}(S_k)$. Transitive dependents are invalidated immediately, while independent parallel steps are preserved idempotently.

3.3 Physical Output Isolation and Sidecar Provenance

Forge enforces a strict tri-file layout:

1. **The contract** (`contract.md`): human-readable stage requirements.
2. **The edit surface** (`output/*`): clean target files carrying zero machine metadata.
3. **The sidecar provenance** (`provenance.json`): out-of-band JSON capturing

$$\mathrm{Provenance}(S_i) = \{\, \mathcal{A}_{\text{pinned}},\ \mathcal{H}_{\text{inputs}},\ \mathcal{C}_{\text{contract\_section}},\ \mathcal{V}_{\text{compiler}} \,\}.$$

4 Calibrated Psychometric Gate Governance

Forge treats evaluation as an empirical measurement problem rather than a single classification call.

4.1 Effort-Axis Sampled Scoring and Variance as Signal

Forge executes N-sample scoring passes across three reasoning effort levels, $\mathcal{E} = \{\text{low}, \text{medium}, \text{high}\}$. For a sample set $S = \{s_1, \ldots, s_N\}$ on check C we define:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} s_i, \qquad \sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(s_i - \mu)^2}, \qquad s_{\min} = \min(S). \tag{2}$$

Two decision invariants follow:

1. **Pass on minimum.** Decision = Pass if $s_{\min} \geq \tau_C$, otherwise Fail. Bounding by the minimum leaves no tolerance for borderline outputs.
2. **Variance halts** (E_UNSTABLE_GATE). If $\sigma > \sigma_{\text{threshold}}$, execution halts. Because the artifact under evaluation is static, dispersion across samples is attributable to the rubric rather than the artifact, which indicates an underspecified rubric and prevents compute being spent on re-rolls.

5 Zero-Trust Cryptographic Attestation

In high-liability environments an assurance offered by the executing party is insufficient. Internal daemon communication uses symmetric HMAC signing; third-party verification requires asymmetric cryptography.

When a run clears its final stage gate, Forge assembles a canonical JSON payload containing the `run_id`, timestamp, input data, final output mutations, and the passing $s_{\min}$ `audit_consistency` scores. Forge signs that payload with **Ed25519** [2, 3], writes the signature to the decision ledger, and exposes the `forge://run/{run_id}/certificate` MCP resource. Third parties, such as CRM systems or external auditors, verify the signature offline against the embedded public key identifier, confirming that the recorded run cleared its declared gates and that the record has not been altered since signing, without Forge ever exposing private signing material.

6 Formal System Invariants

Forge is governed by absolute invariants enforced at runtime and summarised in Table 1.

7 Experimental Verification and Results

The architecture was verified across test suites and live database environments.

- **Cryptographic verification.** 172 database-backed execution proofs and 126 headless schema invariants pass. Asymmetric receipts were minted and verified offline using isolated standalone verification scripts holding no private secrets.
- **Mechanical traceability.** Gate failure injection confirmed that every failure cited the exact rubric clause, with no fabricated citations across the injected set.
- **Sandbox integrity.** Brute-force simulation attempts were neutralised by deterministic input seeding: repeated submissions of an identical payload returned an identical score, removing the re-roll attack surface.

Table 1. Runtime invariants and their enforcement points.

ID	Invariant	Definition	Enforced in
INV-1	Fail-closed preflight	Any schema, path, parity, or hash mismatch halts turn one.	`preflight.py`
INV-2	Transitive staleness	Only steps with altered per-file input digests $\mathcal{D}$ are re-executed.	`staleness.py`
INV-3	Minimum-bound scoring	Gate approval strictly requires $\min(S) \geq \tau$.	`gates/engine.py`
INV-4	Variance bounds	Evaluation variance $\sigma > \sigma_{\max}$ halts execution immediately.	`gates/engine.py`
INV-5	Deterministic sandbox	`simulate_execution` variance is seeded strictly by input payload hash.	MCP server
INV-6	Asymmetric attestation	Certificates must be Ed25519-signed; symmetric keys are forbidden for third-party receipts.	`receipt.ts`

8 Comparative Analysis

Table 2. Forge relative to standard agent frameworks and to ICM.

Dimension	Standard agent frameworks	Interpretable contexts (ICM)	Forge (DCC + MCP)
Execution safety	Blind API execution	Manual human review	Predictive sandbox (`simulate_execution`)
State storage	Memory dictionaries, vector stores	Materialised markdown	Isolated output with sidecar provenance
Staleness tracking	None; full rerun	Step level, coarse	Transitive per-file Merkle digests
Gate evaluation	Single-pass boolean prompt	Single-pass metric score	Effort-sampled distribution ($s_{\min}$, σ)
High variance	Ignored; resolved by chance	Ignored	Halts on rubric ambiguity (E_UNSTABLE_GATE)
Auditability	Internal logging	Filesystem history	Ed25519 cryptographic receipts

Table 2 summarises the comparison. The consistent theme is the relocation of enforcement from post-hoc inspection to pre-execution admission control.

9 Limitations and Threats to Validity

Three limitations bound the claims above and should be read alongside them.

First, a signature establishes integrity and provenance, not correctness. An Ed25519 receipt proves that a specific payload cleared the declared gates and has not been altered since signing. It does not prove that the gates encode the right policy, nor that a passing

action is safe in a sense the rubric failed to capture. The strength of the guarantee is bounded by the quality of the rubric and of the simulation's fidelity to production.

Second, sandbox fidelity is an assumption. Mocked dependencies approximate production behaviour; divergence between the mock and the live system is a residual failure channel that deterministic seeding does not address.

Third, the variance halt distinguishes rubric ambiguity from artifact defect only under the stated condition that the artifact is static across samples. Where sampling temperature, model version, or context assembly varies between passes, dispersion is no longer attributable to the rubric alone.

10 Conclusion

Forge addresses the tension between autonomous LLM capability and deterministic software engineering discipline. By combining Deterministic Context Compilation, Calibrated Psychometric Gate Governance, and MCP-based predictive sandboxing, it removes the fragility of unconstrained agent loops. Ed25519 attestation moves the resulting record from an assurance that must be taken on trust to one that a third party can verify independently and offline, establishing a practical foundation for zero-trust, high-assurance LLM systems in enterprise environments.

References

[1] R. C. Merkle. A digital signature based on a conventional encryption function. *Advances in Cryptology (CRYPTO '87)*, LNCS 293, pp. 369–378, 1988.

[2] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.

[3] S. Josefsson and I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, IETF, January 2017.

[4] S. Rose, O. Borchert, S. Mitchell, and S. Connelly. Zero Trust Architecture. NIST Special Publication 800-207, August 2020.

[5] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems 36*, Datasets and Benchmarks Track, 2023.

[6] Anthropic. Model Context Protocol specification. 2024. `https://modelcontextprotocol.io`

[7] J. Van Clief and D. McDermott. Interpretable Context Methodology: Folder Structure as Agentic Architecture. arXiv:2603.16021, March 2026.

Availability. Source, artifacts, and standalone verification scripts are available at `https://github.com/arcanealgo/forge-research-paper`.